

Student Name	Sarvesh Naik
SRN No	31241943
Roll No	33
Program	Computer Engineering
Year	Third Year
Division	F
Subject	Artificial Intelligence Lab
Assignment No	2

Assignment 2

Problem Statement: Implement the Water Jug problem using the following uninformed search strategies:

a. Depth First Search b. Breadth First Search

Objective:

To implement the Water Jug Problem using uninformed search strategies, namely Depth First Search (DFS) and Breadth First Search (BFS). The objective is to represent the problem as a state space where each state denotes the amount of water in both jugs, and systematically explore possible states without using any heuristic information. DFS is used to explore states deeply along one path at a time, while BFS explores states level by level to find the shortest sequence of steps to reach the target quantity.

What is uninformed search:

Uninformed Search, also known as **Blind Search**, refers to search algorithms that explore the state space without using any additional domain knowledge or heuristic information to guide the search. These algorithms operate only based on the problem definition, such as the initial state, possible actions, goal state, and transition rules.

In uninformed search, the algorithm does not know which state is closer to the goal. It systematically explores all possible states until it finds the solution. The search process is performed using predefined strategies to traverse the state space.

What is Water Jug Problem:

The Water Jug Problem is a classical problem in problem-solving and search techniques. It involves two water jugs with fixed capacities and an unlimited water supply. The objective is to measure a specific target quantity of water using only the given jugs.

Problem Description:

- Two jugs are given with capacities **A** and **B** liters.
- Initially, both jugs are empty.
- The goal is to measure exactly **T liters** of water in either jug.
- You cannot measure directly; you can only perform certain allowed operations.

Allowed Operations:

1. Fill a jug completely.
2. Empty a jug completely.
3. Pour water from one jug into the other until:
 - The first jug becomes empty, or
 - The second jug becomes full.

These operations are repeated systematically to reach the target quantity.

Each state is represented by (x, y)

x = Amount of water in Jug A & y = Amount of water in Jug B

The goal is reached when:

$x = T$ or $y = T$

Water Jug Problem using DFS and BFS:

In the Depth First Search (DFS) approach, the Water Jug problem is solved by exploring one possible sequence of actions deeply before trying alternative paths. DFS is an uninformed search technique because it does not use any heuristic or prior knowledge about the goal state; it simply explores states systematically. DFS follows a Last-In-First-Out (LIFO) principle and is usually implemented using recursion or a stack.

Steps:

1. Start from the initial state (0,0).
2. Mark the state as visited to avoid repetition.
3. Generate all possible next states using allowed operations:
 - Fill Jug A
 - Fill Jug B
 - Empty Jug A
 - Empty Jug B
 - Pour $A \rightarrow B$
 - Pour $B \rightarrow A$
4. Choose the first possible state and move deeper.
5. Continue exploring deeper states until:
 - The target is found, or
 - No further moves are possible.
6. If a dead end is reached, backtrack to the previous state and try another path.

If DFS reaches a state where:

- All possible moves are already visited, and
- The target is not achieved

Then it backtracks and explores another branch.

Pseudo code:

DFS(state, visited):

if state is goal:

return SUCCESS

add state to visited

for each successor S of state:

if S not in visited:

set parent[S] = state

result = DFS(S, visited)

if result == SUCCESS:

return SUCCESS

return FAILURE

BFS(initial_state):

create empty queue Q

enqueue initial_state into Q

create visited set

add initial_state to visited

while Q is not empty:

current = dequeue(Q)

if current is goal:

return SUCCESS

for each successor S of current:

if S not in visited:

add S to visited

set parent[S] = current

enqueue S into Q

return FAILURE

Code:

```
import sys, uuid, math

import matplotlib.pyplot as plt

import matplotlib.patches as mpatches
```

```

import matplotlib.path_effects as pe

from matplotlib.patches import FancyArrowPatch

# --- INPUT ---

print("=" * 42)

print("    WATER JUG - DFS Solver")

print("=" * 42)

cap_a      = int(input("    Jug A capacity : "))

cap_b      = int(input("    Jug B capacity : "))

target     = int(input("    Target amount  : "))

jug_choice = input("    Target in Jug? (A/B): ").strip().upper()

while jug_choice not in ("A", "B"):

    jug_choice = input("    Enter A or B: ").strip().upper()

def is_goal(s):

    return s[0] == target if jug_choice == "A" else s[1] == target

# --- SUCCESSORS ---

def successors(state):

    a, b = state

    moves = []

    if a < cap_a: moves.append((cap_a, b), "Fill A")

    if b < cap_b: moves.append((a, cap_b), "Fill B")

    if a > 0:     moves.append((0, b),     "Empty A")

    if b > 0:     moves.append((a, 0),     "Empty B")

```

```

    if a > 0 and b < cap_b:

        p = min(a, cap_b-b); moves.append((a-p, b+p), "A-B"))

    if b > 0 and a < cap_a:

        p = min(b, cap_a-a); moves.append((a+p, b-p), "B-A"))

    return [(s, l) for s, l in moves if s != state]

# — BUILD TREE —————

# This builds a proper TREE (not a graph) where:

# - Each node is a UNIQUE tree position (state can repeat = pruned node)

# - "pruned" nodes = state already visited on this path = DFS BACKTRACKS here

# - Tree stops as soon as goal is first found (as real DFS would)

#

# This is the standard textbook representation of a DFS search tree.

sys.setrecursionlimit(50000)

nodes = {} # node_id -> dict

root_id = "root"

nodes[root_id] = {

    "state": (0, 0), "depth": 0, "parent": None,

    "label": "", "status": "start", "children": []

}

goal_id = [None]

stop = [False]

visit_counter = [0]

```



```

def build_tree(node_id, visited, depth=0):

    if stop[0]:

        return

    state = nodes[node_id]["state"]

    visited = visited | {state}    # copy - each path has its own visited set

    for nxt, lbl in successors(state):

        if stop[0]:

            return

        cid = str(uuid.uuid4())[:8]

        if nxt in visited:

            # Already on this path → dead end / pruned → DFS backtracks here

            nodes[cid] = {

                "state": nxt, "depth": depth+1, "parent": node_id,

                "label": lbl, "status": "pruned", "children": [],

                "visit_num": None

            }

            nodes[node_id]["children"].append(cid)

        else:

            visit_counter[0] += 1

            status = "goal" if is_goal(nxt) else "new"

            nodes[cid] = {

                "state": nxt, "depth": depth+1, "parent": node_id,

```

```

        "label": lbl, "status": status, "children": [],

        "visit_num": visit_counter[0]

    }

    nodes[node_id]["children"].append(cid)

    if status == "goal":

        goal_id[0] = cid

        stop[0] = True

        return

    build_tree(cid, visited, depth+1)

nodes[root_id]["visit_num"] = 0

build_tree(root_id, set(), 0)

if goal_id[0] is None:

    print("No solution found."); sys.exit()

# Mark the solution path from goal back to root

sol_ids = set()

nid = goal_id[0]

while nid is not None:

    sol_ids.add(nid)

    nid = nodes[nid]["parent"]

for nid in sol_ids:

    if nodes[nid]["status"] not in ("start", "goal"):

```

```

        nodes[nid]["status"] = "solution"

goal_depth = nodes[goal_id[0]]["depth"]

total_nodes = len(nodes)

pruned_count = sum(1 for n in nodes.values() if n["status"] == "pruned")

print(f"\n Goal found at depth : {goal_depth}")

print(f" Tree nodes total      : {total_nodes}")

print(f" Pruned (backtracks) : {pruned_count}")

# --- LAYOUT -----

# Post-order: leaves get sequential x, parents centred above children

def assign_x(nid, counter):

    ch = nodes[nid]["children"]

    if not ch:

        nodes[nid]["x"] = counter[0] * 1.0

        counter[0] += 1

        return

    for c in ch:

        assign_x(c, counter)

    xs = [nodes[c]["x"] for c in ch]

    nodes[nid]["x"] = (min(xs) + max(xs)) / 2.0

assign_x(root_id, [0])

```

```

# y = -depth

for nid, nd in nodes.items():

    nd["y"] = -nd["depth"] * 1.8


# Scale x a bit for readability

all_x = [nd["x"] for nd in nodes.values()]

x_span = max(all_x) - min(all_x) + 1

for nd in nodes.values():

    nd["x"] = nd["x"] * max(1.5, 12.0 / x_span)


# — DRAW —————

all_x2 = [nd["x"] for nd in nodes.values()]

all_y2 = [nd["y"] for nd in nodes.values()]

W = max(18, (max(all_x2) - min(all_x2) + 4))

H = max(14, abs(min(all_y2)) + 4)

fig, ax = plt.subplots(figsize=(W, H))

fig.patch.set_facecolor("#0f0f1a")

ax.set_facecolor("#0f0f1a")


# Colours

C_START = "#FF8C00"    # orange

C_SOL   = "#FFD700"    # gold — solution path

C_GOAL  = "#00E676"    # green

```

```

C_NEW      = "#4A6FA5"    # blue - explored (not on solution path)

C_PRUNED   = "#B71C1C"    # dark red - pruned/backtracked

C_SOLEDGE  = "#FFD700"

C_NEWEDGE  = "#4A6FA5"

C_PRUNEDGE = "#B71C1C"

C_BT_ARROW = "#FF4081"    # bright pink - backtrack arrows


NODE_R = 0.45    # node radius for positioning arrow endpoints


def node_color(status):

    return {

        "start":    C_START,

        "solution": C_SOL,

        "goal":     C_GOAL,

        "new":      C_NEW,

        "pruned":   C_PRUNED,

    }.get(status, C_NEW)


# — 1. Draw edges —————

for nid, nd in nodes.items():

    if nd["parent"] is None:

        continue

    par = nodes[nd["parent"]]

    x0, y0 = par["x"], par["y"]

    x1, y1 = nd["x"], nd["y"]

```

```

is_sol      = (nid in sol_ids and nd["parent"] in sol_ids)

is_pruned   = nd["status"] == "pruned"

color = C_SOLEDGE if is_sol else (C_PRUNEDGE if is_pruned else C_NEWEDGE)

lw         = 2.5 if is_sol else 1.2

ls         = "solid" if (is_sol or not is_pruned) else (0, (6, 3))

ax.annotate("",

            xy=(x1, y1 + NODE_R), xytext=(x0, y0 - NODE_R),

            arrowprops=dict(

                arrowstyle="-|>", color=color, lw=lw,

                linestyle=ls, mutation_scale=14,

                connectionstyle="arc3,rad=0.0"),

            zorder=2)

# Operation label on edge

ex = (x0 + x1) / 2 + 0.08

ey = (y0 + y1) / 2

ax.text(ex, ey, nd["label"], fontsize=7, color=color,

        ha="center", va="center", zorder=4,

        bbox=dict(boxstyle="round,pad=0.15",

                    fc="#0f0f1a", ec=color, lw=0.6, alpha=0.9))

```

— 2. Backtrack arrows —————

```

# For every pruned node: draw a curved red arrow from that pruned

# node back UP to its parent to show "DFS came here, can't go further,

# so it backtracks to parent and tries the next sibling"

#

# The arrow is drawn as a curve that goes LEFT of the edge so it

# doesn't overlap with the downward tree edge.


for nid, nd in nodes.items():

    if nd["status"] != "pruned":

        continue

    par = nodes[nd["parent"]]

    x0, y0 = nd["x"], nd["y"] # FROM pruned child

    x1, y1 = par["x"], par["y"] # TO parent

    ax.annotate("",

        xy=(x1, y1 - NODE_R), # arrive at parent (bottom)

        xytext=(x0, y0 + NODE_R), # depart pruned node (top)

        arrowprops=dict(

            arrowstyle="-|>",

            color=C_BT_ARROW,

            lw=1.8,

            linestyle=(0, (5, 3)),

            mutation_scale=13,

            alpha=0.9,

            connectionstyle="arc3,rad=0.5"),

```

```

        zorder=5)

# — 3. Draw nodes —————

for nid, nd in nodes.items():

    x, y = nd["x"], nd["y"]

    status = nd["status"]

    col = node_color(status)

    sz = (1800 if status in ("goal", "start") else

          1500 if status == "solution" else

          1100 if status == "new" else 900)

    # Glow for important nodes

    if status in ("start", "solution", "goal"):

        ax.scatter(x, y, s=sz+700, c=col, alpha=0.15, zorder=3)

    ax.scatter(x, y, s=sz, c=col, zorder=6,

               edgecolors="#ffffff25", linewidths=0.9)

    # State label

    tc = "#000000" if status in ("start","solution","goal") else "#FFFFFF"

    ax.text(x, y, f"({nd['state'][0]}, {nd['state'][1]})",

            ha="center", va="center",

            fontsize=8.5, fontweight="bold", color=tc, zorder=7)

    # Visit-order number for non-pruned nodes

```



```

vnum = nd.get("visit_num")

if vnum is not None:

    vc = (C_GOAL if status == "goal" else

          C_SOL if status == "solution" else

          C_START if status == "start" else

          "#90CAF9")

    ax.text(x - 0.58, y + 0.58, f"#{vnum}",

            fontsize=9.5, fontweight="bold", color=vc,

            ha="center", va="center", zorder=8,

            path_effects=[pe.withStroke(linewidth=2.5,

                                         foreground="#0f0f1a")])

# ✖ on pruned nodes

if status == "pruned":

    ax.text(x + 0.52, y + 0.52, "✖",

            fontsize=11, color=C_PRUNED,

            ha="center", va="center", fontweight="bold", zorder=8,

            path_effects=[pe.withStroke(linewidth=2,

                                         foreground="#0f0f1a")])

# START / GOAL top badges

for nid, txt in [(root_id, "START"), (goal_id[0], "GOAL ★")]:

    nd = nodes[nid]

    bg = C_START if txt == "START" else C_GOAL

    ax.text(nd["x"], nd["y"] + 0.75, txt,

```

```

        ha="center", va="center",

        fontsize=9, color="#000000", fontweight="bold", zorder=9,

        bbox=dict(boxstyle="round,pad=0.35",

                    fc=bg, ec=bg, lw=1.8))

# --- LEGEND -----

legend_handles = [

    mpatches.Patch(color=C_START, label="Start node (0,0)",

    mpatches.Patch(color=C_SOL, label="Solution path (DFS found this route)",

    mpatches.Patch(color=C_GOAL, label=f"Goal - {target}L in Jug {jug_choice}"),

    mpatches.Patch(color=C_NEW, label="Explored node (off solution path)",

    mpatches.Patch(color=C_PRUNED, label="✖ Already visited → DFS must
backtrack"),

    mpatches.Patch(color=C_SOLEDGE, label="—— Solution path edge"),

    mpatches.Patch(color=C_NEWEDGE, label="— DFS forward edge"),

    mpatches.Patch(color=C_BT_ARROW, label="↖ Backtrack (DFS retreats to parent)",

]

ax.legend(handles=legend_handles, loc="lower left",

          facecolor="#1a1a2e", edgecolor="#444466",

          labelcolor="white", fontsize=9.5,

          framealpha=0.97, borderpad=1.0, handlelength=1.6)

# --- TITLE -----

ax.set_title(

    f"DFS - Depth First Search | Water Jug Problem\n"

```

```

f"Jug A = {cap_a}L    Jug B = {cap_b}L    Target = {target}L in Jug {jug_choice}\n"

f"DFS dives deep into each branch first • "

f"When it hits ✖ (already visited state), it backtracks ↶ to parent and tries the
next child • "

f"Solution found at depth {goal_depth} • {pruned_count} backtracks",

color="white", fontsize=12, fontweight="bold",

loc="left", pad=16, linespacing=1.75)

ax.axis("off")

plt.savefig("dfs_tree.png", dpi=160, bbox_inches="tight",

          facecolor=fig.get_facecolor())

print(" Saved → dfs_tree.png")

plt.show()

def get_successors(state):

    a, b = state

    moves = []

    if a < cap_a:
        moves.append(((cap_a, b), "Fill A"))

    if b < cap_b:
        moves.append(((a, cap_b), "Fill B"))

    if a > 0:
        moves.append(((0, b), "Empty A"))

    if b > 0:
        moves.append(((a, 0), "Empty B"))

    if a > 0 and b < cap_b:
        p = min(a, cap_b-b); moves.append(((a-p, b+p), "A→B"))

    if b > 0 and a < cap_a:
        p = min(b, cap_a-a); moves.append(((a+p, b-p), "B→A"))

```

```

    return [(s, l) for s, l in moves if s != state]

def bfs():

    """Standard BFS - returns tree info + BFS visit order."""

    start = (0, 0)

    visited = {start}

    parent = {start: None}

    elabel = {}

    depth = {start: 0}

    order = [start]          # BFS discovery order

    queue = deque([start])

    goal = None

    while queue:

        state = queue.popleft()

        if is_goal(state):

            goal = state

            break

        for nxt, lbl in get_successors(state):

            if nxt not in visited:

                visited.add(nxt)

                parent[nxt] = state

                elabel[(state, nxt)] = lbl

                depth[nxt] = depth[state] + 1

```

```

        order.append(nxt)

        queue.append(nxt)

    return parent, elabel, depth, goal, order

parent, elabel, depth_map, goal_state, visit_order = bfs()

if goal_state is None:

    print("\n ❌ No solution found. Try different values.")

    exit()

# Reconstruct solution path

path, node = [], goal_state

while node is not None:

    path.append(node); node = parent[node]

path.reverse()

path_set = set(path)

path_edges = set(zip(path, path[1:]))

steps = len(path) - 1

print(f"\n ✅ BFS Solution: {steps} steps (guaranteed SHORTEST!)")

for i, s in enumerate(path): print(f"      Step {i}: {s}")

```

```
# -----  
  
# 3. BUILD GRAPH + WIDE LEVEL LAYOUT  
  
# -----  
  
G = nx.DiGraph()  
  
for child, par in parent.items():  
  
    G.add_node(child)  
  
    if par is not None:  
  
        G.add_edge(par, child, label=elabel.get((par, child), ""))  
  
  
level_groups = defaultdict(list)  
  
for n, d in depth_map.items():  
  
    level_groups[d].append(n)  
  
  
pos = {}  
  
for d, nodes in level_groups.items():  
  
    nodes_sorted = sorted(nodes)  
  
    for i, n in enumerate(nodes_sorted):  
  
        x = (i - (len(nodes_sorted)-1) / 2) * 3.2  
  
        y = -d * 2.8  
  
        pos[n] = (x, y)  
  
  
max_depth = max(depth_map.values())  
  
max_width = max(len(v) for v in level_groups.values())  
  
# -----
```

```

# 4. FIGURE – wide canvas + info panel on right

# -----

fig_w = max(32, max_width * 3.6)

fig_h = max(22, (max_depth + 1) * 3.2 + 4)

fig = plt.figure(figsize=(fig_w, fig_h), facecolor="#07090e")

gs = gridspec.GridSpec(1, 2, width_ratios=[3.2, 1], wspace=0.02)

ax = fig.add_subplot(gs[0]) # tree

ap = fig.add_subplot(gs[1]) # info panel

ax.set_facecolor("#07090e")

ap.set_facecolor("#0a1520")

ap.set_xlim(0,1); ap.set_ylim(0,1); ap.axis("off")

# --- Colour palette (OCEAN / WATER theme – BFS = ripples in water)

C_BG      = "#07090e"

C_START   = "#00b4d8" # bright cyan      – start

C_EXPL    = "#0d3047" # dark teal        – explored off-path

C_PATH    = "#ffd166" # warm gold          – solution path nodes

C_GOAL    = "#06d6a0" # mint green         – goal

C_PEDGE   = "#ef476f" # rose                – solution path edges

C_OEDGE   = "#0d2030" # very dim navy       – other edges

C_WAVE    = "#0077b6" # deep ocean blue    – wave band

# -----

```

```

# 5. HORIZONTAL WAVE BANDS ◀ THE KEY BFS VISUAL

# Every level = one "wave ring" expanding outward

# -----

all_x = [p[0] for p in pos.values()]

xmin, xmax = min(all_x)-2.5, max(all_x)+2.5

for d in range(max_depth+1):

    y = -d * 2.8

    alpha = 0.10 + 0.06*(d % 2)

    # Shaded horizontal band

    ax.fill_between([xmin, xmax], [y-1.35, y-1.35], [y+1.35, y+1.35],

                    color=C_WAVE, alpha=alpha, zorder=0)

    ax.axhline(y - 1.35, color=C_WAVE, lw=0.5, ls=":", alpha=0.3)


    # Wave label LEFT

    label = f"≈ Wave 0 (Start)" if d == 0 else f"≈ Wave {d}"

    ax.text(xmin - 0.3, y, label,

            color=C_WAVE, fontsize=9, va="center", alpha=0.65,

            style="italic", ha="right",

            path_effects=[pe.withStroke(linewidth=2, foreground="#07090e")])


    # Node count RIGHT

    n_wave = len(level_groups[d])

    ax.text(xmax + 0.3, y,

            f"{n_wave} node{'s' if n_wave!=1 else ''}",

```



```

        color=C_WAVE, fontsize=8.5, va="center", alpha=0.6,

        style="italic", ha="left")

# -----

# 6. DRAW EDGES

# -----

for u, v in G.edges():

    on_path = (u, v) in path_edges

    ec = C_PEDGE if on_path else C_OEDGE

    lw = 2.8      if on_path else 0.8

    ax.annotate("", xy=pos[v], xytext=pos[u],

                arrowprops=dict(arrowstyle="-|>", color=ec, lw=lw,

                                mutation_scale=18,

                                connectionstyle="arc3,rad=0.06"),

                zorder=2)

# Edge labels

for (u, v), lbl in elabel.items():

    if (u, v) not in G.edges(): continue

    mx = pos[u][0]*0.45 + pos[v][0]*0.55

    my = pos[u][1]*0.45 + pos[v][1]*0.55

    on_path = (u, v) in path_edges

    fc2 = "#2a0014" if on_path else "#071525"

    ec2 = C_PEDGE    if on_path else "#1e3d5c"

```

```

ax.text(mx + 0.08, my, lbl, fontsize=7, color=ec2,

        ha="center", va="center", zorder=5,

        bbox=dict(boxstyle="round,pad=0.22", fc=fc2, ec=ec2, lw=0.8, alpha=0.92))

# -----

# 7. DRAW NODES

# -----

visit_idx = {n: i for i, n in enumerate(visit_order)}

for n in G.nodes():

    x, y = pos[n]

    if n == goal_state:

        color, size = C_GOAL, 2000

    elif n in path_set:

        color, size = C_PATH, 1500

    elif n == (0, 0):

        color, size = C_START, 1500

    else:

        color, size = C_EXPL, 1000

    # Glow halo for important nodes

    if n in path_set or n == goal_state or n == (0, 0):

        ax.scatter(x, y, s=size+700, c=color, alpha=0.15, zorder=3)

```

```

ax.scatter(x, y, s=size, c=color, zorder=4,

          edgecolors="#ffffff25", linewidths=0.9)

# State text (A, B)

tc = "#000000" if n in (path_set | {goal_state}) else "#8fc9df"

ax.text(x, y, f"({n[0]},{n[1]})", ha="center", va="center",

        fontsize=8, fontweight="bold", color=tc, zorder=7)

# Visit-order badge on off-path nodes

if n not in path_set and n != goal_state:

    ax.text(x, y + 0.62, f"#{visit_idx.get(n, '?')}",

            ha="center", va="center",

            fontsize=5.5, color="#4da6c8", zorder=8, alpha=0.75)

# Mini jug fill bars below solution-path nodes

if n in path_set or n == goal_state:

    bar_ox = x - 0.52

    for ji, (jval, jcol, jcap) in enumerate([

        (n[0], "#00b4d8", cap_a),    # Jug A - blue

        (n[1], "#ff9f1c", cap_b)]): # Jug B - orange

        fill_h = 0.55 * (jval/jcap) if jcap > 0 else 0

        ax.add_patch(plt.Rectangle(

            (bar_ox + ji*0.58, y-1.1), 0.44, 0.58,

            fc="#112233", ec="#223344", lw=0.6, zorder=6))

    if fill_h > 0:

```

```

        ax.add_patch(plt.Rectangle(

            (bar_ox + ji*0.58, y-1.1), 0.44, fill_h,

            fc=jcol, alpha=0.75, zorder=7))

    ax.text(bar_ox + ji*0.58 + 0.22, y-1.12 + fill_h + 0.07,

            f"{'A' if ji==0 else 'B'}={jval}",

            ha="center", va="bottom", fontsize=5.5,

            color=jcol, fontweight="bold", zorder=8)

# -----

# 8.  START / GOAL BADGES

# -----

def badge(ax, x, y, txt, bg, fg):

    ax.text(x, y+0.95, txt, ha="center", va="center", fontsize=8.5,

            color=fg, fontweight="bold", zorder=10,

            bbox=dict(boxstyle="round,pad=0.35", fc=bg, ec=fg, lw=1.6))

badge(ax, pos[(0,0)][0], pos[(0,0)][1], "► START", C_START, "#000")

badge(ax, pos[goal_state][0], pos[goal_state][1], "★ GOAL!", C_GOAL, "#000")


# -----

# 9.  TITLE

# -----

ax.set_title(

```

```

"BFS - Breadth First Search\n"

"\nLike ripples on water: every state 1 step away is checked before any state 2
steps away\n\n"

f"Jug A = {cap_a}L | Jug B = {cap_b}L | Target = {target}L in Jug {jug_choice}
| "

f"Solution = {steps} steps (SHORTEST guaranteed ✓)",

color="white", fontsize=13, fontweight="bold",

loc="left", pad=16, linespacing=1.6)

ax.axis("off")

# -----

# 10. SIDE PANEL

# -----

def pt(y, txt, sz=9, col="#cde8f5", bold=False, mono=False):

    ap.text(0.07, y, txt, transform=ap.transAxes,

            fontsize=sz, color=col, fontweight="bold" if bold else "normal",

            va="top", ha="left",

            family="monospace" if mono else "DejaVu Sans", clip_on=False)

# Panel border

ap.add_patch(plt.Rectangle((0.03, 0.01), 0.94, 0.97, transform=ap.transAxes,

                            fc="#081420", ec=C_WAVE, lw=1.5, alpha=0.9, zorder=0))

y = 0.97

pt(y, "HOW BFS WORKS", 12, C_START, True); y -= 0.05

```

```

for line in ["Drop a stone in a calm pond.",

            "Ripples spread outward, ring",

            "by ring, in all directions.",

            "", "BFS does the same thing:"]:

    pt(y, line, 8.5, "#8fc9e0"); y -= 0.028

for line in [" 1. Check ALL states 1 op away",

            " 2. Then ALL 2 ops away",

            " 3. Then 3 ops... and so on."]:

    pt(y, line, 8, "#c0dde8"); y -= 0.026

y -= 0.005

pt(y, "→ First goal found is the", 8.5, "#8fc9e0"); y -= 0.026

pt(y, "  SHORTEST PATH POSSIBLE!", 9.5, C_GOAL, True); y -= 0.042


pt(y, "TREE SHAPE", 10, C_PATH, True); y -= 0.030

for line in ["WIDE & FLAT", "Many nodes per level.",

            "Each shaded band = one BFS wave."]:

    pt(y, line, 8.5, "#8fc9e0"); y -= 0.026

y -= 0.012


pt(y, "COLOUR GUIDE", 10, C_START, True); y -= 0.032

legend_items = [

    (C_START, "Start state  (0, 0)"),

    (C_PATH,  "Solution path node"),

    (C_GOAL,  f"GOAL  ({target}L in Jug {jug_choice})"),

    (C_EXPL,  "Explored, off-path"),

```

```

(C_PEDGE, "Solution path edge"),

(C_OEDGE, "Other tree edge"),

]

for bg, desc in legend_items:

    ap.add_patch(plt.Rectangle((0.07, y-0.021), 0.11, 0.027,

                               transform=ap.transAxes, fc=bg, ec="#ffffff33", lw=0.7, zorder=5))

    pt(y, f"          {desc}", 8, "#cde8f5"); y -= 0.032

y -= 0.008


pt(y, "SOLUTION PATH", 10, C_GOAL, True); y -= 0.030

pt(y, f"{steps} steps — provably optimal", 8.5, C_PATH); y -= 0.030


for i, (a, b) in enumerate(path):

    if y < 0.07: break

    bar_a = "█" * a + "░" * (cap_a - a)

    bar_b = "█" * b + "░" * (cap_b - b)

    col    = C_GOAL if (a,b)==goal_state else (C_START if i==0 else C_PATH)

    op     = ""

    if i < len(path)-1:

        op = " ← " + elabel.get((path[i], path[i+1]), "")

    pt(y, f" {i}) A[{bar_a}] B[{bar_b}]{op}", 7.2, col, mono=True)

    y -= 0.028

y -= 0.012


pt(y, "STATS", 10, C_START, True); y -= 0.028

```

```

plt(y, f"Nodes in tree : {len(visit_order)}", 8.5, "#8fc9e0"); y -= 0.024

plt(y, f"Solution depth: {steps}", 8.5, "#8fc9e0"); y -= 0.024

plt(y, f"Tree levels : {max_depth+1}", 8.5, "#8fc9e0"); y -= 0.024

plt(y, f"Max nodes/wave: {max_width}", 8.5, "#8fc9e0")

# Footer

fig.text(0.5, 0.003,

        "BFS visits every node at depth D before any node at depth D+1 "

        "- this is why it ALWAYS finds the SHORTEST path",

        ha="center", fontsize=9.5, color=C_WAVE, style="italic")

plt.savefig("bfs_tree.png", dpi=160, bbox_inches="tight",

           facecolor=fig.get_facecolor())

print("\n  Saved → bfs_tree.png")

plt.show()

```

Output:

```

-----
Jug A capacity : 4
Jug B capacity : 3
Target amount  : 2
Target in Jug? (A/B): A

```


DFS — Depth First Search | Water Jug Problem

Jug A = 4L Jug B = 3L Target = 2L in Jug A

DFS dives deep into each branch first • When it hits ✕ (already visited state), it backtracks ↩ to parent and tries the next child • Solu



✓ BFS Solution: 6 steps (guaranteed SHORTEST!)

Step 0: (0, 0)

Step 1: (4, 0)

Step 2: (1, 3)

Step 3: (1, 0)

Step 4: (0, 1)

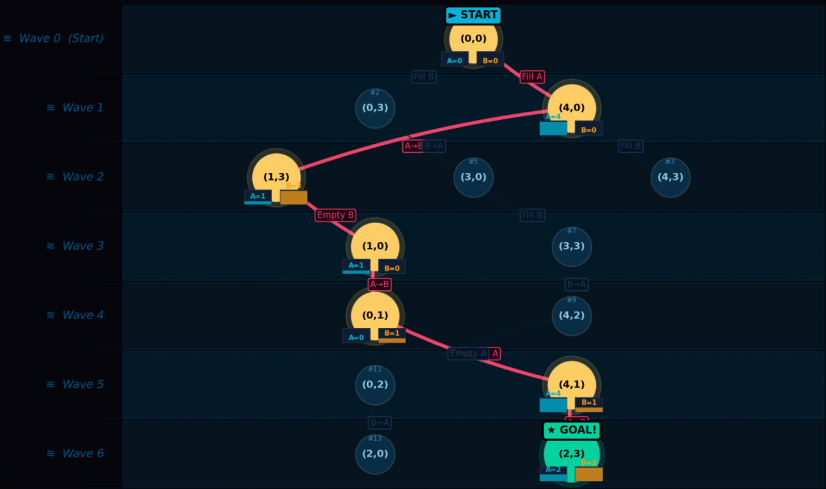
Step 5: (4, 1)

Step 6: (2, 3)



Saved → bfs_tree.png

BFS — Breadth First Search
"Like ripples on water: every state 1 step away is checked before any state 2 steps away"
Jug A = 4L | Jug B = 3L | Target = 2L in Jug A | Solution = 6 steps (SHORTEST guaranteed ✓)



HOW BFS WORKS

Drop a stone in a calm pond.
Ripples spread outward, ring by ring, in all directions.

BFS does the same thing:
1. Check ALL states 1 op away
2. Then ALL 2 ops away
3. Then 3 ops... and so on.

→ First goal found is the **SHORTEST PATH POSSIBLE!**

TREE SHAPE

WIDE & FLAT
Many nodes per level.
Each shaded band = one BFS wave.

COLOUR GUIDE

- Start state (0, 0)
- Solution path node
- GOAL (2L in Jug A)
- Explored, off-path
- Solution path edge
- Other tree edge

SOLUTION PATH

6 steps — provably optimal

- 0) A[] B[] → Fill A
- 1) A[] B[] → A-B
- 2) A[] B[] → Empty B
- 3) A[] B[] → A-B
- 4) A[] B[] → Fill A
- 5) A[] B[] → A-B

STATS

Nodes in tree : 14
Solution depth: 6
Tree levels : 7
Max nodes/wave: 3

BFS visits every node at depth D before any node at depth D+1 — this is why it ALWAYS finds the SHORTEST path